

ChargeEV

ChargeEV — Current State Report

Bugs, errors, missing data, and functionality status

Generated by re-running every check against the code as it stands right now, not summarizing from memory. Every claim below has evidence behind it.

TL;DR

There are zero known bugs, broken imports, syntax errors, or failing tests in the code right now. Everything that can be verified automatically passes. What remains is not bugs — it's a short list of *known, deliberate gaps* (things not yet built, or not verifiable without a browser), all documented below so nothing is hidden.

If you're deploying: the one thing that genuinely needs your attention is **#1 in the "Environment" section** — the backend can't fully boot in a sandbox without internet access to Prisma's servers. On a normal machine this is a non-issue.

1. Automated health checks — all green ✓

Run just now, fresh:

Check	Result	What it proves
Backend test suite (<code>npm test</code>)	73/73 passing, 5/5 suites	Core booking/auction/payment/admin logic works
Backend ESLint	0 problems	No undefined vars, unused/missing imports
Frontend ESLint	0 problems	Same, across all <code>.jsx</code>
Frontend build (<code>npm run build</code>)	succeeds	Every <code>.jsx</code> parses & bundles — no syntax/bracket errors
Import resolver	313/313 resolve,	No missing files or wrong-case

(custom)	case-exact	paths (the bug that crashes only on Linux)
Backend <code>node --check</code> (every <code>.js</code>)	all parse	No missing brackets/parens anywhere
<code>npm audit</code> (backend)	0 vulnerabilities	—
TODO/FIXME/HACK markers in source	none	No half-finished code left with a marker

Conclusion: static code analysis found nothing to fix. No missing imports, no undefined variables, no path/case errors, no bracket errors.

2. Functionality status — what’s verified vs. what isn’t

The backend exposes **62 API route handlers** across 11 controllers (`admin`, `ai`, `auth`, `bid`, `booking`, `complaint`, `ev`, `review`, `slot`, `station`, `verification`). Here’s the honest breakdown of what’s actually *proven* to work vs. merely *present*:

✓ **Verified working (backed by a live-passing test)**

The e2e suite exercises the full critical path end to end: - Admin bootstrap, registration + login for **every** role, role guards - Station lifecycle (owner creates → admin approves) - Time-window bookings **with overlap/race-condition rejection** - Check-in cost-locking, Stripe webhook idempotency (pays exactly once), completion - Full auction flow (bidding, priority scoring, closing, auto-booking) - Admin dashboard + audit trail, complaints inbox, verified-purchase reviews - Logout revoking the token server-side

⚠ **Present but NOT directly tested (works in code-trace, no assertion)**

These aren’t *broken* — they’re just not covered by an automated test, so “working” rests on code inspection, not a green checkmark: - **AI recommendation endpoint** (`ai.controller`) — no test touches it - **Email OTP happy path** (`verification.controller`) — validation errors are tested, but a real successful OTP verification isn’t (flagged back in Phase 7.1 too) - **Live Socket.IO delivery** — the auction *logic* is tested, but actual real-time socket push to a connected client isn’t (needs a live server) - **EV CRUD and slot management** — used as test *setup*, but their own endpoints aren’t independently asserted

⊘ **Cannot be verified in this environment at all (no live**

browser)

- Whether pages render without **browser-console errors**
 - Real click-through **404 testing** across navigation
 - **Mobile responsiveness** at real viewport widths (only ever code-audited)
 - Whether the hero/design **looks right** visually
-

3. Known gaps & deliberate incomplete items (not bugs)

These are things *intentionally* not finished yet — tracked, not forgotten:

1. **DRY pass / component-extraction** (Phase 8.2) — the last code-quality checklist item; hasn't had its dedicated pass. Not a bug, just unfinished polish.
 2. **Bootstrap removal** (Phase 10) — in progress and *deliberately paused*. The replacement `grid.css` is written but **not imported anywhere yet**, and Bootstrap is **still active** (`main.jsx` still imports it). So the app is in a consistent state (fully on Bootstrap) — the migration simply hasn't started touching pages. See `PHASE_10_BOOTSTRAP_REMOVAL_PLAN.md`.
 3. **Optional Dockerfile** (Phase 8.3) — not created. Vercel/Render don't need it.
 4. **~11 hardcoded hex colors** in components (star ratings, status badges in `Stars.jsx`, `Stations.jsx`, `Navbar.jsx`, `OwnerDashboard.jsx`, `AIRecommend.jsx`) sit outside the shared CSS-variable palette. Not *wrong* — they render fine — but they're a consistency smell if you want full theme centralization.
 5. **Sentry & Google Analytics are wired but dormant** — they no-op until you add a real `SENTRY_DSN` / `VITE_GA_MEASUREMENT_ID`. That's by design, but worth remembering they're not actually collecting anything yet.
-

4. Environment / deployment caveats

1. ⚠ **Backend won't fully boot in a no-internet sandbox.** Prisma's query engine binary downloads during `npm install (prisma generate)`, and that download is blocked in this sandbox. On your machine or any normal deploy box it works fine. The code now fails *gracefully* here — `npm run doctor` and the startup path both explain the cause instead of dumping a raw stack trace — but **you should confirm the backend boots on a real machine before calling deployment verified**. This is the single most important item to check.
2. **Frontend esbuild / vite vulnerability** (1 moderate, 1 high, dev-only) — pre-existing since Phase 6. The fix requires a **major Vite version bump** (breaking change), deliberately left out of scope. It affects the dev server only, not production output.

3. **No real database in this sandbox** — all tests use an in-memory mock, so nothing here has run against actual MongoDB. `npm run doctor` will verify a real connection once you point `DATABASE_URL` at one.
-

5. Bottom line

- **Bugs / errors / broken code right now:** none found. Every automated check passes.
- **Missing data:** no orphaned/half-built code; the “missing” items are the *deliberately unfinished* ones in §3 (DRY pass, Bootstrap migration, optional Dockerfile) — all tracked.
- **Is everything working?** Everything *testable* is working (73/73). The honest caveat is that a meaningful slice of functionality (AI recommend, OTP happy-path, live sockets, and anything visual/browser-side) is verified only by code inspection, not by an automated test or a real browser — because neither a browser nor a live DB exists in this environment.

Recommended next actions, in priority order: 1. Boot the backend on a real machine + DB (`npm run doctor` , then `npm start`) — confirms the one thing this sandbox can’t. 2. Open the app in a real browser with DevTools — clears the entire “can’t verify visually” section at once. 3. Then, if you want: finish the DRY pass, or resume the Bootstrap migration with the pilot-page approach from the Phase 10 plan.